Compilers

Guillermo A. Pérez

Lecture X: Error recovery, type checking, and other semantic analyses

TL;DR: This lecture in short

How do we get the most out of our AST?

We have already constructed a symbol table using the AST. We are now interested in obtaining more semantic information about the program by analyzing the AST.

Main references

- **Introduction to Language Theory and Compilers.** Gilles Geeraerts and G. A. Pérez. 2019.
- **Crafting a Compiler.** Charles N. Fischer et al. 2010. Pearson Education.
- **Compiler Design.** Helmut Seidl et al. 2012. Springer
- **Compilers: Principles, Techniques, Tools.** Alfred Aho et al. 2006. Pearson Education.
- Online: Wikipedia, LLVM, ANTLR, ...

Required and target competences

What tools do we need?

Formal language theory (languages and machines, machines and computability); graph algorithms

What skills will we obtain?

- theory: type systems, reachability analysis, ...
- practice: how to automatically check some semantical properties of the input program

How will these skills be useful?

- It will allow you to understand the limits of what your compiler can do.

Outline

1. Top-down error recovery
2. Semantical analysis
3. Type checking
4. Reachability and termination
5. Exceptions
6. Procedure/function calls

Error recovery and repair

Recovery and repair

When a single error in the input program is located, error messages regarding the rest of the program may still be relevant. But how does one process the rest?

- Error recovery: essentially, ignore the current error; have the parser reach a configuration from which it can read the rest of the input
- Error repair: the parser attempts to fix the input so as to obtain a valid input prefix and then continue

Panic mode

The parser skips input tokens until it finds a frequently occurring delimiter (e.g. a semicolon).

Errors in LL(1) parsers

Detection

Remember: LL(1) parsers are based on a single look-ahead. If no entry in the action table is enabled by the look-ahead, we have an error. We can then output a message explaining what the parser expected (see top of stack).

Recovery in recursive descent

Simple idea for a panic mode recovery:

- Every call to a non-terminal-function is given as argument some set of potential end-of-block lexical units
- If an error occurs, then the function discards tokens until one such token is matched (e.g. semicolon)

Example: simple LL(1) error recovery

```
1  def S(termset):
2      n = get_next_character()
3      # gewoon parsing
4      # code goes here
5      if found_error:
6          while n not in termset:
7              n = get_next_character()
8          return True
```

Semantic analysis

Type checking

Type correctness is the essence of semantic analysis. We traverse the AST to verify that the types of all components at every node conform to the expected type for the programming language.

- E.g: Condition-expressions in a conditional statement should evaluate to a Boolean type.

Other semantic analyses

Reachability and termination

Languages such as Java require that the code be checked for instructions that can never be reached. We can also verify that constructs terminate normally (cf. returns, breaks, etc.).

Exceptions

Exception handling is common to all modern programming languages. The convention is that thrown exceptions must be caught or listed in the construct's throw list.

Type checking

Types as invariants

Assigning a type t to a variable x is, in essence, an invariant.

- Throughout the execution of the program, x will only hold values of type t.

Turing-complete, hence undecidable

Most modern programming languages are Turing complete. So type checking is undecidable!

```
1  void main(){
2      int x;
3      ...  // some super cool code
4      x = "am␣I␣an␣int?";
5      ...  // more awesome code
6  }
```

Static type checking

Static type checking

Henceforth, we write type checking for static type checking: apply inference rules to deduce types of expressions (without caring about reachability of instructions). This approach is

- conservative: some programs will be marked as not correct, yet the type-incorrect instructions may not be executed ever.

Static type consistency

Type consistency

A single bottom-up traversal of an expression tree should allow us to determine the type of the expression and check whether

- the constants
- variables
- and operators

are used consistently with their types! Constant types are built into the grammar, variable types can be found in the symbol table.

Operators

Operators

We can check whether the types of the provided arguments are correct. If that is the case, the return type gives the type of the operator application.

Overloading and implicit casting

Which operator is applied to $5 + 3 * 2$?

- $+ : \mathbb{Z} \times \mathbb{Z} \to \mathbb{Z}$
- $+ : \mathbb{R} \times \mathbb{R} \to \mathbb{R}$

What about $5.0 + 3 * 1.0$?

Language-specific subtleties

Implicit casting

Some languages support (automatic) casting:

- integers into real numbers,
- integers into Booleans, or
- subclass into superclasses.

This requires keeping a hierarchy of types in memory during the type-analysis traversal of an expression.

Overloaded operators

Some languages support operators whose return type depends on the types of the arguments. To type check them, all overloaded versions of the operator must be tried — respecting the potential type hierarchy.

- In some languages operator selection depends on the return/assignment type too!

Typechecker implementation

Tasks

The checker must

- verify the validity of the expression given the types of the descendants and
- derive the type of parent expressions (propagation).

Implementation

- It is typically implemented as a bottom-up traversal of the AST using the symbol table.
- Statements have no type, so they start the recursive traversal of their sub-trees.

Reachability and termination

General reachability is . . . undecidable

Once more, for Turing-powerful programming languages, deciding reachability amounts to deciding the halting problem. So we focus on conservative approximations.

Statements are the focus

In this semantical analysis, we focus on whether (lists of) statements

- are reachable,
- terminate normally, i.e. control flow "falls through".

Reachability & termination examples

```
1   ...; return; a = a + 1; ...
```

Examples

- In the above code, the increment of *a* is unreachable
- break, return, ..., instructions terminate abnormally

Rules for reachability and termination

For a simple analysis, follow these rules

Consider a statement list $S = s_1, \ldots, s_n$.

1. If S is reachable then s_1 is reachable.
2. If s_n terminates abnormally then so does S.
3. If S is the body of a function or procedure then S is reachable.
4. Local variable declarations, assignments, function calls, memory allocation, increment, decrement instructions all terminate normally.
5. A null statement or statement list does not generate error messages if it is not reachable. However, they propagate their non-reachability status to their successors.
6. If a statement has a predecessor then it is reachable if its predecessor terminates normally.

Example

```
1  void example() {
2      int v; v++; return; ; v = 10; v = 20;
3  }
```

Reachable and terminating statements

- The function's body is reachable.

Example

```
1  void example() {
2      int v; v++; return; ; v = 10; v = 20;
3  }
```

Reachable and terminating statements

- The function's body is reachable.
- The declaration of v is reachable.

Example

```
1  void example() {
2      int v; v++; return; ; v = 10; v = 20;
3  }
```

Reachable and terminating statements

- The function's body is reachable.
- The declaration of v is reachable.
- Both the declaration and increment terminate normally.

Example

```
1  void example() {
2      int v; v++; return; ; v = 10; v = 20;
3  }
```

Reachable and terminating statements

- The function's body is reachable.
- The declaration of v is reachable.
- Both the declaration and increment terminate normally.
- The return statement is reachable but does not terminate normally.

Example

```
1  void example() {
2      int v; v++; return; ; v = 10; v = 20;
3  }
```

Reachable and terminating statements

- The function's body is reachable.
- The declaration of v is reachable.
- Both the declaration and increment terminate normally.
- The return statement is reachable but does not terminate normally.
- The null statement that follows is unreachable, so the assignment of 10 to v is also unreachable.

Example

```
1  void example() {
2      int v; v++; return; ; v = 10; v = 20;
3  }
```

Reachable and terminating statements

- The function's body is reachable.
- The declaration of v is reachable.
- Both the declaration and increment terminate normally.
- The return statement is reachable but does not terminate normally.
- The null statement that follows is unreachable, so the assignment of 10 to v is also unreachable.
- It can generate an error message about being unreachable!

Example

```
1  void example() {
2      int v; v++; return; ; v = 10; v = 20;
3  }
```

Reachable and terminating statements

- The function's body is reachable.
- The declaration of v is reachable.
- Both the declaration and increment terminate normally.
- The return statement is reachable but does not terminate normally.
- The null statement that follows is unreachable, so the assignment of 10 to v is also unreachable.
- It can generate an error message about being unreachable!
- However, it does terminate normally.

Example

```
1  void example() {
2      int v; v++; return; ; v = 10; v = 20;
3  }
```

Reachable and terminating statements

- The function's body is reachable.
- The declaration of v is reachable.
- Both the declaration and increment terminate normally.
- The return statement is reachable but does not terminate normally.
- The null statement that follows is unreachable, so the assignment of 10 to v is also unreachable.
- It can generate an error message about being unreachable!
- However, it does terminate normally.
- The last assignment is reachable (and normally terminating).

Break

Coffee? 10min break.

Analyzing control-flow instructions

Remark

Note that functions should never terminate normally. In fact, if the body of a function terminates normally, there is an error: it is not returning a value.

If-then-else

The AST for a conditional statement should have (at least) three sub-trees:

- the condition expression,
- the block/statement-list to execute if the expression is true,
- the block/statement-list to execute if the expression is false.

For reachability/termination analysis, we assume the condition can be both true and false.

- So both statement-lists are reachable.

Example

```
1   if (cond) a = 1; else a = 2;
```

Example

Our semantical analysis passes would

1. first check `cond`, and in particular verify that it has type `bool`,
2. both branches of the conditional are then checked, and
3. if they both complete normally, then the conditional terminates normally.

Analyzing loops (1/2)

While loop

The AST for a `while` loop should have (at least) two sub-trees:

- the condition expression and
- the block/statement-list to execute if the expression is true.

Constant conditions

If it evaluates to the constant true then the while-statement is marked as abnormally terminating — since it is an infinite loop. If it evaluates to false the statement-list is marked as unreachable.

Analyzing loops (2/2)

The body of the loop

If the statement-list contains a `break` and the `while` was marked as abnormally terminating because of a true-constant condition, then we reset this to normally terminating.

- If the body is reachable then it is semantically analyzed otherwise normally.
- If the body terminates normally, then the `while` statement terminates normally (unless it is an infinite loop with no break).

While example

```
1   while (i % 10 != 0) {
2       a[i--] = 0;
3   }
```

Analysis

1. The condition is not a constant expression, so the body is reachable.
2. Assignments terminate normally, so the body also terminates normally.
3. The `while` statement thus terminates normally as well.

Still a conservative analysis

Exercise

Can you modify the previous C snippet so that it still terminates normally and the body is reachable while, semantically, the loop is infinite?

Still a conservative analysis

Exercise

Can you modify the previous C snippet so that it still terminates normally and the body is reachable while, semantically, the loop is infinite?

Exercise

Can you modify it so that the body of the loop does not terminate normally while semantically the loop is finite? (Tip: use an if with a return in one branch.)

Continue, break, return

Continue, break

We must verify that it is contained in the body of a loop.

Break

It marks the immediately containing loop statement as normally terminating if it is an infinite loop.

Continue, break, return

All three terminate abnormally.

Exception-handling

First thing

For languages that support exceptions and try-catch control flow, you should

- Extend your symbol table so that functions and procedures have a list of possible exceptions they can throw as type information.

Throw/raise

Thrown exceptions are almost like returned values. The enclosing function or procedure should be able to throw them if you find a `throw` instruction.

Try and catch

Try-catch statements (for Java)

All catch-clause statement-lists should be reachable.

1. We check exception types in catch-clauses do not "hide" each other.
   - A general exception type in a first catch-clause will always be used instead of other ones with more specific exception types.
2. While analyzing the try body, we compile a list of possible exceptions.
   - They should enable all catch-clauses.
   - The subset of checked exceptions should be covered by all catch-clauses and the type of the enclosing procedure.
3. Finally, all catch-clause bodies are also analyzed.

Checking procedure calls

The simple case

For procedure calls, as for operators, we must keep in mind

- whether the language allows overloading
- the types of the arguments of the call
- the name of the procedure
- (sometimes the return-type of the procedure)

Matching formal and actual parameters

When is a procedure definition applicable to a call?

1. Their number should match.
2. The parameters are bindable (in general: the type of the formal parameter allows it to be assigned the actual parameter).

Calling the most specific possibility

```
1  class A { void M(A param) { ... } }
2  class B
3    extends A { void M(B param) { ... } }
4  class C { void M() { ... } }
5  class D extends C { void M() { ... } }
```

Examples

- If we call `M()` in an instance of class `D` we would like to use its definition of `M`.
- If we call `M(b)` from class `B` where `b` is of type `B` we would prefer to use the definition of the method in class `B` because it is a closer match

Conclusions

- Static type checking is the main task carried out during semantic analysis
- It can be seen as a very basic form of verification of a program.
- Reachability of instructions and their effect on the control flow is also something that can be approximated
- Correct exception handling and function-call matching is a complicated task required to compile modern OO languages
- Many of these tasks are treated in more detail in the field of formal verification